

String

doesn't support assignment

s = "python",
s[0] = 'j'

Page No.	
Date	

Type

1) String Data Type

- A sequence of characters (alphabets, numbers or special characters) enclosed within quotes.

eg: (String supports indexing & slicing)

name = "Prince"

- It takes duplicate character & ordered data type.

Types of Quotes in String

1. Single quotes

2. Double quotes

3. Single triple quotes

4. Double triple quotes

1. Single quotes ('')

For single line (Simple string)
when no special formatting is needed.

eg: a = 'good morning'

2. Double quotes ("")

For simple strings (one line)

Helpful when string contains single quote inside. (apostrophe)

eg:

a = "it's great"

a = 'it's great' # error

3. Triple Single quotes :- (''')

For multi-line strings
or commenting purpose we are using
eg:-

```
msg = ''' Hello
        Welcome
        '''
```

It will comment the code)

4. Double Triple quotes :- ("""")

Same as triple single quotes
commenting purpose.

* How to create empty string using normal way
or by using object.

→ Normal way a = ' ' (externally)

using object a = string() a = str() (internally)

* String data type (internally represent as an object
str())

* String data type (externally represent as using
quotes)

1) How to check length of string.
Using len() function.

Syntax:-
len(variable_name)

len(string)

2) What is meaning of len function?

→ It will return number of characters present inside string and it will consider space also.

eg: S = "Google"
len(S) #6

Methods:-

String (30)

- | | | |
|-------------------|--------------------|----------------------|
| 1) title() | • 14) isupper() | • 27) removesuffix() |
| 2) Capitalize() | • 15) islower() | • 28) isidentifier() |
| 3) Upper() | • 16) isspace() | • 29) iskeyword() |
| 4) lower() | 17) index() | • 30) istitle() |
| 5) Swapcase() | 18) rindex() | |
| 6) Count() | 19) split() | |
| • 7) ends with() | 20) rsplit() | |
| • 8) Startswith() | 21) Strip() | |
| 9) replace() | 22) rstrip() | |
| 10) join() | 23) lstrip() | |
| • 11) isalpha() | 24) find() | |
| • 12) isalnum() | 25) rfind() | |
| • 13) isdigit() | 26) removeprefix() | |

list :- (13) []

- | | | |
|-------------|--------------------|------------|
| 1) append() | 7) count() | 13) sort() |
| 2) extend() | 8) index() | |
| 3) insert() | 9) reverse() | |
| 4) pop() | 10) copy() | |
| 5) remove() | 11) shallow copy() | |
| 6) clear() | 12) deepcopy() | |

Tuple :- (2) ()

- | | |
|------------|------------|
| 1) count() | 2) index() |
|------------|------------|

Dictionary :- (11) { : }

- | | | |
|--------------------|---------------|-----------------------------------|
| 1) get() | 6) update() | 11) values() |
| 2) To access value | 7) clear() | 12) items() |
| 3) setdefault() | 8) copy() | 13) key [key] = value. |
| 4) pop() | 9) fromkeys() | del keyword |
| 5) popitem() | 10) keys() | |

Sets :- (17) { }

- | | | |
|----------------|---------------------------|-----------------------------------|
| 1) add() | 7) intersection() | 15) copy() |
| 2) update() | 8) symmetric-difference() | 16) difference-update |
| 3) clear() | 9) difference() | 17) intersection-update() |
| 4) remove() | 10) isdisjoint() | 18) symmetric-difference-update() |
| 5) discard() | 11) issuperset() | |
| 6) pop() | 12) issubset() | |
| 7) del keyword | 14) union() | |

Method Name

Syntax

Example

1) index()

Syntax:

vn.index('Substeing', Si, Ei+1)

optional

a = "Good Morning"

a.index('G')

o/p: 0

a = "banana"

a.index('a')

o/p: 1

a.index('c')

o/p: Value Error

2) rindex

Syntax:

vn.rindex('Substeing', Si, Ei+1)

a = "The moon is white"

a.rindex('e')

o/p: 16

1st occurrence.

3) find()

Syntax:

vn.find('Substeing', Si, Ei+1)

a.find('m')

o/p: 4

a.find('o')

o/p: 5

a.find('M')

o/p: -1

4) rfind()

Syntax:

vn.rfind('Substeing', Si, Ei+1)

a.rfind('m')

o/p: 4

a.rfind('o')

o/p: (6)

Explanation

- Forward moves from left to right
- It is used to find the position of substring/character.
- When substring is absent we get value error
- work same as find()
- Character $\rightarrow$ position
- Takes 1st occurrence. if we want second or 3rd occurrence position then pass Si & Ei .

- Backward from right to left but we start indexing from 0
- Starts checking from ending.
- Value error.
- last occurrence

- For searching character or substring. position.
 $\text{C} \rightarrow \text{P}$

- Forward
- 1st occurrence, if substring is not present then by default gives output as -1.

- Backward $\text{C} \rightarrow \text{P}$

- -1 default o/p
- Same as rindex. but gives no error.
- last occurrence.

* Indexing

The process of extracting / fetching single character in a string is called indexing.

Indexing

Forward

Indexing

- left to right
- Starts from (0)

Backward

Indexing

- right to left
- starts from (-1)

Syntax:-

vn [index position | number]

Note:

If the specified position is not present it will throw index error.

(P) → (C)

eg: "python programming"

h[2]

h[6]

h[-3]

h[-6]

h[100]

o/p: 't'

' '

'i'

'a'

Index error

String index out of range

* Slicing

The process of fetching multiple character is called slicing.

Types:-

- 1) Positive slicing
- 2) Negative slicing
- 3) Combination (+ve + -ve)

1) Positive slicing :-

Syntax :-
$$vn[start\ index : end\ index + 1 : step\ value]$$

 By default it will take complete length of string
 By default always 0
 By default it will one

Why end index + 1 ?

→ Because it will include previous character if we didn't mention +1 it will exclude the previous character.

eg:

x : "be active in class"

evitca a = evitca → x[10 : 16 : 1]

ni a = x[-7 : -9 : -1]

class x[13 : 18 : 1]

2) Negative slicing :-

Syntax :-

$$vn[-si : -ei - 1 : -sv]$$

 ↑ Default Values -1 ↑ length of string ↑

It is mandatory field -1

Note :- In backward slicing stepvalue is mandatory thing.

* dict()

For finding all attributes present inside a data type.

Syntax :- `dict(data type)`

* String Methods

1) upper()

It will convert all the lowercase to uppercase.

Syntax :-

`vn.upper()` or `String.upper()`

2) lower()

It will convert all uppercase to lowercase.

Syntax :-

`vn.lower()`

3) Swapcase()

It will swap character (upper to lower and vice versa).

Syntax :-

`vn.swapcase()`

4) Capitalize()

Convert 1st character/letter of the sentence to "uppercase" followed by lowercase.

Syntax :-

`vn.capitalize()`

title()

It converts first character/letter of a word and after space any word convert into upper case

Syntax :-

vn.title()

Count()

It will give count of specified character substring inside a string

③ → ⑩

If value is not present it gives 0

Syntax :-

vn.count("substring", [si] [ei])

Eg :-

x = "good morning"

x.count("o")

3

x.count("n")

2

x.count("o", 4, 7)

x.count("o", 0, 4)

2

x.count("z")

0

replace()

It is used to replace the character

Syntax :-

vn.replace("Old value", "New value", [count])

eg:

a = "yellow"

a.replace("l", "*")

o/p :- Ye**ow

a.replace("l", "*", 1)

o/p :- Ye*low

How many
time we
want to
change

8) .isalnum()

It is combination of alphabets & numbers.

It returns True if string alphanumeric / alphabets / numbers.

Syntax :-

vn.isalnum()

9) .isalpha()

It is written True when string contains only alphabets. otherwise False. (Upper / lower)

Syntax :-

vn.isalpha()

if 'This is my'

False no spacing.

10) .isdigit()

Only numbers True

otherwise False

Syntax :-

vn.isdigit()

11) .isupper()

It will return True if string contains only upper case characters

Numbers and special characters allowed

No lowercase

Syntax :-

vn.isupper()

12) .islower()

It will return True if lowercase characters are present otherwise False

Syntax :-

vn.islower()

3). isspace()

It will return True if all characters in string are space / empty, otherwise False.

Syntax:-

vn.isspace()

eg:- s = " hello"

b = " "

s.isspace()

b.isspace()

False

True

4). istitle()

Returns true if 1st letter of word in uppercase & after space every 1st letter in uppercase. Otherwise, False.

Syntax:-

vn.istitle()

eg:

a = "The Moon Is"

a = "The moon is"

a.istitle()

a.isupper() a.istitle()

True

False

5). index()

It will return position (index) of specific value. Forward.

If value is not present, return value error. (Indexing start from zero)

Syntax:-

vn.index ("Substring", [si] [ei])

a = "The Moon"

a.index('o')

5

16) index()

It will return position of specific value
 value error if position of that character is
 not present
Backward.

Syntax:-

Vn.index("substring", [si] [ei])

17) find() (from zero)

It will return position index of specified value
 it will return -1 if specified value not present.

Syntax:-

Vn.find("substring", [si] [ei])

© → P

18) rindex()

It will return position of specified value.
 it will return -1 if the character is not
 present. Backward

Syntax:-

Vn.rindex("substring", [si] [ei])

19) removeprefix()

Remove the prefix (starting part) from string
 only if it matches.

Syntax:-

Vn.removeprefix('character')

a = "good" a.removeprefix('G') good.

if character not present gives same as
 input

20) removeSuffix()

Remove given suffix (ending part)

Syntax:

vn.removeSuffix('character')

eg = a = "Python"

a.removeSuffix("n")

O/p: 'Pytho'

21) startswith()

It will return true if string is starting with specified characters, otherwise False.

Syntax:

String.startswith("substring", [si], [ei])
vn.startswith("substring", [si], [ei])

eg:

a = "The moon is white"

a.startswith("The")

True

a.startswith("m", 4, 5)

True

22) endswith()

It returns True if the string ends with specified characters, otherwise false.

Syntax:

vn.endswith("substring", [si], [ei])

eg:-

a = "The moon is"

a.endswith("is")

True

23) .join()

iterable

all collection data types are iterables.
(str, list, tuple, set, dict)

It will join elements in between only not ending & starting.

Syntax:-

"String".join(iterable)

eg:-

a = "good morning" , 'boop' ,

'-'.join(a)

o/p: g-o-o-d-m-o-r-n-i-n-g

['boop']

24) .strip()

It removes any leading and trailing characters

Syntax:-

vn.strip('characters')

25) .rstrip()

It remove any trailing character (right side)

Syntax:- vn.rstrip('character')

26) .lstrip()

It removes any leading character (left)

Syntax:-

vn.lstrip('character')

eg: s = "#@hello@#"

s.lstrip("#@")

o/p: hello@#

27) .split()

It split the string at specified separator & returns a list forward.

By default split at space.

Syntax:-

`vn.split("separator", maxsplit)`

eg: "good morning"

`a.split()`

`['good', 'morning']`

`a.split('m')`

`['good', 'orning']`

`a.split("o", 2)`

`['g', 'd', 'orning']`

28) .rsplit()

It split strings at specified separator & returns a list (Backward)

By default Split at space.

Syntax:-

`vn.rsplit("separator", maxsplit)`

Note:-

1) If you don't specify the max split in rsplit the output will look same as split method.

2) Converts string into a list.